

ReqManager

Sistema de Gestão de Requisições Internas

Especificação e Documentação Técnica

Nome: Benildo Pedro

Empresa: Contactus

Data: Julho 2026

Índice

1.Introdução.....	4
1.1 Contexto.....	4
1.2 Objetivo do Documento	4
1.3 Âmbito do Sistema.....	4
2. Requisitos Do Sistema	5
2.1 Módulo de Autenticação (Auth)	5
2.2 Módulo de Gestão de Recursos	5
2.3 Módulo de Requisições	6
2.4 Módulo de Controlo de Estados e Transições.....	6
2.5 Módulo de Segurança e Validações	6
2.6 Requisitos Não Funcionais (RNF)	7
2.7 Regras de Negócio (RN)	7
3.Arquitetura da Solução	7
3.1 Visão Geral.....	8
3.2 Decisões Técnicas.....	8
3.3 Camadas do Backend.....	8
3.4 Camadas do Frontend.....	8
3.5 Fluxo de Autenticação	8
4. Diagramas UML.....	9
4.1 Tabela de casos de uso	9
4.2 Diagrama de Sequência.....	11
4.3 Diagrama de Atividades	12
5.Diagrama Entidade-Relação.....	13
6. API REST	13
6.1 Convenções Gerais	13
6.2 Códigos de Resposta HTTP.....	13
6.3 Autenticação.....	14
6.4 Recursos	14
6.5 Requisições	14
6.6 Aprovação	14
7. Segurança.....	15
7.1 Autenticação e Autorização.....	15
7.2 Proteção de Dados	16
7.2.1 Passwords	16
7.2.2 Variáveis de ambiente	16
7.2.3 Exposição de dados	16
7.3 Prevenção de Vulnerabilidades.....	16
8. Docker	17

8.1 Arquitetura de Containers	17
9. Guia de Instalação	17
9.1 Pré-requisitos.....	17
9.2 Passos de Instalação.....	17
9.3 Verificação da Instalação	18
9.4 Credenciais de Teste	18
9.5 Paragem do Sistema.....	18
10. Conclusão.....	19

1.Introdução

1.1 Contexto

O presente documento constitui a especificação técnica e funcional do sistema ReqManager, desenvolvido no âmbito de um projeto de estágio na Contactus. O sistema insere-se na evolução contínua da plataforma RHevolution e de outros produtos internos destinados à gestão organizacional.

O ReqManager foi concebido como um módulo de Gestão de Requisições Internas, com o objetivo de centralizar e simplificar o processo de pedido e aprovação de recursos organizacionais partilhados, tais como espaços, equipamentos e viaturas.

1.2 Objetivo do Documento

Este documento tem como objetivo descrever de forma completa e estruturada todos os aspetos do sistema desenvolvido, nomeadamente:

- Os requisitos funcionais organizados por módulo
- A arquitetura técnica da solução e as decisões tomadas
- Os diagramas de modelação (UML e ER)
- A documentação da API REST
- As medidas de segurança implementadas
- A infraestrutura Docker e o guia de instalação

Destina-se a servir como referência técnica para avaliação do projeto, instalação do sistema e eventual evolução futura da solução.

1.3 Âmbito do Sistema

O ReqManager permite a gestão centralizada de requisições de recursos internos através de uma aplicação web acessível por browser. O sistema suporta dois perfis de utilizador, Administrador e Colaborador, com responsabilidades e permissões distintas.

O âmbito funcional do sistema cobre as seguintes áreas:

- Autenticação e controlo de acesso por perfil
- Gestão do catálogo de recursos (espaços, equipamentos, viaturas e outros)
- Criação, consulta e cancelamento de requisições por parte dos colaboradores
- Aprovação, rejeição e conclusão de requisições por parte dos administradores
- Controlo de estados e transições do ciclo de vida de uma requisição
- Validação de regras de negócio e segurança em todas as operações

Ficou fora do âmbito deste projeto, por decisão técnica justificada, a gestão de contas de utilizador via interface, as contas são criadas através de um seeder de base de dados no arranque do sistema.

2. Requisitos Do Sistema

2.1 Módulo de Autenticação (Auth)

Responsável por identificar o utilizador e definir o seu nível de acesso (Administrador ou Colaborador), servindo de base para todas as autorizações dos restantes módulos.

<i>ID</i>	<i>Requisito Funcional</i>
------------------	-----------------------------------

<i>RF01</i>	O sistema deve permitir login com email e password
<i>RF02</i>	O sistema deve permitir logout, invalidando a sessão/token
<i>RF03</i>	O sistema deve atribuir um perfil (Administrador ou Colaborador) a cada utilizador autenticado
<i>RF04</i>	O sistema deve bloquear o acesso a qualquer funcionalidade sem autenticação válida
<i>RF05</i>	O sistema deve emitir um token de sessão válido após autenticação bem-sucedida

Nota de implementação - Gestão de contas de utilizador

Não está prevista, no âmbito deste projeto, a criação de uma interface de gestão de utilizadores (criação, edição ou desativação de contas via aplicação). Esta decisão decorre da análise dos requisitos do enunciado, que não especifica esta funcionalidade como obrigatória, permitindo assim concentrar o esforço de desenvolvimento nas funcionalidades centrais do sistema: gestão de recursos, requisições e aprovação.

As contas de utilizador necessárias para demonstração e teste do sistema (um Administrador e dois a três Colaboradores) serão criadas através de um *seeder* da base de dados, executado automaticamente no arranque do ambiente (via Docker/migrations). Esta abordagem garante que o sistema fica imediatamente operacional após a instalação, sem necessidade de registo manual.

2.2 Módulo de Gestão de Recursos

Responsável pelo catálogo de recursos disponíveis para requisição (espaços, equipamentos, viaturas, outros), incluindo o seu ciclo de vida administrativo.

<i>ID</i>	<i>Requisito Funcional</i>
------------------	-----------------------------------

<i>RF06</i>	O Administrador deve poder criar um recurso (nome, tipo, descrição)
<i>RF07</i>	O Administrador deve poder editar os dados de um recurso
<i>RF08</i>	O Administrador deve poder eliminar um recurso
<i>RF09</i>	O Administrador deve poder ativar/desativar um recurso
<i>RF10</i>	O Colaborador deve poder consultar a lista de recursos ativos
<i>RF11</i>	O sistema deve permitir filtrar recursos por tipo e por estado

2.3 Módulo de Requisições

Responsável pela criação e gestão dos pedidos de utilização de recursos por parte dos colaboradores.

ID	Requisito Funcional
RF12	O Colaborador deve poder criar uma requisição, indicando recurso, data de início e data de fim
RF13	O Colaborador deve poder consultar as suas próprias requisições
RF14	O Administrador deve poder consultar todas as requisições do sistema
RF15	O Colaborador deve poder cancelar uma requisição da sua autoria
RF16	O sistema deve impedir o cancelamento de requisições já concluídas
RF17	O sistema deve permitir filtrar requisições por estado

2.4 Módulo de Controlo de Estados e Transições

Responsável por definir e garantir o ciclo de vida correto de uma requisição, assegurando que apenas transições de estado válidas ocorrem.

ID	Requisito Funcional
RF18	O sistema deve atribuir o estado "Pendente" a toda nova requisição criada
RF19	O sistema deve permitir a transição de "Pendente" para "Aprovada" ou "Rejeitada"
RF20	O sistema deve permitir a transição de "Aprovada" para "Concluída" ou "Cancelada"
RF21	O sistema deve permitir a transição de "Pendente" para "Cancelada"
RF22	O sistema deve impedir transições de estado inválidas (ex: de "Concluída" para "Pendente")
RF23	O sistema deve apresentar um dashboard ao administrador com estatísticas globais do sistema

2.5 Módulo de Segurança e Validações

Responsável por garantir que toda a ação realizada no sistema respeita as permissões do utilizador e as regras de negócio do domínio, independentemente da camada de apresentação.

ID	Requisito Funcional
RF24	O sistema deve impedir que um Colaborador aceda a funcionalidades exclusivas de Administrador
RF25	O sistema deve impedir que um Colaborador cancele requisições de outros utilizadores
RF26	O sistema deve impedir a criação de requisições com datas passadas
RF27	O sistema deve impedir a criação de requisições sobrepostas para o mesmo recurso
RF28	O sistema deve impedir a requisição de recursos inativos
RF29	O sistema deve armazenar as passwords de forma cifrada (hash)
RF30	O sistema deve validar o formato e a obrigatoriedade dos campos em todos os formulários/endpoints

RF31 | O sistema deve restringir a aprovação/rejeição de requisições exclusivamente a Administradores

2.6 Requisitos Não Funcionais (RNF)

ID **Requisito Não Funcional**

RNF01	Sistema deve correr em containers Docker (frontend, backend, BD)
RNF02	API deve seguir princípios REST
RNF03	Passwords devem ser armazenadas com hash (nunca em texto simples)
RNF04	Interface responsiva (desktop e mobile)
RNF05	Validações de negócio aplicadas no backend (não confiar só no frontend)
RNF06	Tempo de resposta da API < 1s em operações normais

2.7 Regras de Negócio (RN)

ID **Regras de Negócio**

RN01	Não pode haver sobreposição de datas para o mesmo recurso
RN02	Apenas recursos ativos podem ser requisitados
RN03	Não são permitidas requisições com datas passadas
RN04	Apenas o autor pode cancelar a sua requisição
RN05	Apenas administradores aprovam/rejeitam

3.Arquitetura da Solução

3.1 Visão Geral

O sistema foi desenvolvido seguindo uma arquitetura cliente-servidor, com separação clara entre o frontend e o backend. O frontend é uma Single Page Application (SPA) desenvolvida em Angular, que comunica com o backend através de uma API REST desenvolvida em Laravel. A base de dados utilizada é MySQL, e toda a infraestrutura é gerida através de Docker.

3.2 Decisões Técnicas

<i>Tecnologia</i>	<i>Decisão</i>	<i>Justificação</i>
Laravel	Framework backend	Fornecer autenticação, ORM, migrações e validação prontos a usar, reduzindo o tempo de desenvolvimento
Angular	Framework frontend	Framework robusto para SPA com suporte nativo a componentes, routing e formulários reativos
MySQL	Base de dados	Base de dados relacional open source, compatível com Laravel e adequada para dados estruturados
Sanctum	Autenticação	Solução oficial do Laravel para autenticação de SPA via tokens, simples e segura
Docker	Infraestrutura	Garante que o sistema funciona de forma idêntica em qualquer ambiente

3.3 Camadas do Backend

O backend segue uma arquitetura em camadas, com responsabilidades claramente separadas:

Controllers - Recebem os pedidos HTTP, delegam a validação para os Form Requests e a lógica para os Services. São mantidos propositalmente simples.

Form Requests - Responsáveis pela validação dos dados de entrada. Cada operação tem o seu próprio Form Request, garantindo que o controller só é executado com dados válidos.

Models - Representam as entidades da base de dados (User, Resource, Request) e as suas relações, utilizando o ORM Eloquent do Laravel.

Middleware/Guards - Responsáveis pela autenticação (Sanctum) e autorização (verificação de perfil admin/colaborador) antes de qualquer operação ser executada.

3.4 Camadas do Frontend

O frontend segue uma arquitetura modular baseada em feature modules:

Core - Contém os serviços globais (AuthService, ResourceService, RequestService), o interceptor HTTP que injeta automaticamente o token em todos os pedidos, e os guards de rota que protegem páginas conforme o perfil do utilizador.

Features - Cada módulo funcional tem a sua própria pasta (auth, resources, requests, approval), com os seus componentes independentes.

Shared - Componentes reutilizáveis entre módulos, como a Navbar.

3.5 Fluxo de Autenticação

1. Utilizador submete email + password
2. Angular envia POST /api/login

3. Laravel valida credenciais
4. Laravel devolve token + dados do utilizador
5. Angular guarda token em localStorage
6. Interceptor injeta token em todos os pedidos seguintes
7. Laravel valida token em cada pedido protegido via middleware auth:sanctum

4. Diagramas UML

4.1 Tabela de casos de uso

ID	Caso de Uso	Ator(es)	Descrição	Pré-condições	Pós-condições
UC01	Login	Colaborador, Administrador	Autenticar-se no sistema com email e password	Utilizador registado e ativo	Sessão/token válido criado; acesso ao perfil correspondente
UC02	Logout	Colaborador, Administrador	Terminar a sessão ativa	Utilizador autenticado	Sessão/token invalidado
UC03	Consultar recursos	Colaborador	Listar e filtrar recursos ativos disponíveis	Utilizador autenticado	Lista de recursos ativos apresentada
UC04	Criar requisição	Colaborador	Submeter um pedido de uso de um recurso, indicando datas	Utilizador autenticado; recurso ativo selecionado	Requisição criada com estado "Pendente"
UC05	Consultar requisições	Colaborador	Ver as suas próprias requisições e respetivo estado	Utilizador autenticado	Lista de requisições do utilizador apresentada
UC06	Cancelar requisição	Colaborador	Cancelar uma requisição da sua autoria	Requisição pertence ao utilizador; estado $\neq$ Concluída	Requisição passa a estado "Cancelada"

UC07	Gerir recursos	Administrador	Criar, editar, eliminar e ativar/desativar recursos	Utilizador autenticado como Administrador	Catálogo de recursos atualizado
UC08	Consultar todas as requisições	Administrador	Ver todas as requisições do sistema, com filtros	Utilizador autenticado como Administrador	Lista global de requisições apresentada
UC09	Aprovar requisição	Administrador	Aprovar uma requisição pendente	Requisição em estado "Pendente"	Requisição passa a estado "Aprovada"; histórico atualizado
UC10	Rejeitar requisição	Administrador	Rejeitar uma requisição pendente, com observações	Requisição em estado "Pendente"	Requisição passa a estado "Rejeitada"; observações registadas; histórico atualizado
UC11	Criar utilizador	Administrador	Criar uma conta (Colaborador ou Administrador), com nome, email, password inicial e perfil	Utilizador autenticado como Administrador	Nova conta criada e disponível para login
UC12	Editar utilizador	Administrador	Alterar dados de um utilizador (nome, email, perfil)	Utilizador autenticado como Administrador	Dados do utilizador atualizados

UC13	Ativar/desativar utilizador	Administrador	Bloquear ou reativar o acesso de um utilizador	Utilizador autenticado como Administrador	Utilizador deixa de conseguir (ou volta a conseguir) fazer login
------	-----------------------------	---------------	--	---	--

4.2 Diagrama de Sequência

Este diagrama de sequência cobre os dois fluxos principais: criação da requisição (com o ramo de erro de validação) e aprovação/rejeição pelo admin (com o ramo de transição inválida).



4.3 Diagrama de Atividades

Temos o diagrama de atividades, focado no ciclo de vida completo da requisição.



5. Diagrama Entidade-Relação

O diagrama apresenta o modelo relacional do sistema, com as entidades users, resources e requests, e os respectivos atributos, chaves e restrições.



6. API REST

6.1 Convenções Gerais

Base URL: `http://localhost:8000/api`

Formato de dados: JSON

Autenticação: Bearer Token (Sanctum)

Todos os pedidos protegidos requerem o header `Authorization: Bearer <token>`

Todos os pedidos devem incluir o header `Accept: application/json`

6.2 Códigos de Resposta HTTP

CÓDIGO	SIGNIFICADO
--------	-------------

200	Operação bem sucedida
201	Recurso criado com sucesso
401	Não autenticado - token inválido ou ausente
403	Não autorizado - perfil sem permissão
404	Recurso não encontrado
422	Erro de validação - dados inválidos

6.3 Autenticação

Login

Método: POST | Endpoint: /api/login | Autenticação: Não requerida

CAMPO	TIPO	OBRIGATÓRIO	DESCRIÇÃO
EMAIL	string	Sim	Email do utilizador
PASSWORD	string	Sim	Password do utilizador

Resposta 200: devolve objeto user (id, name, email, role) e token de sessão.

Logout

Método: POST | Endpoint: /api/logout | Autenticação: Requerida

Resposta 200: {"message": "Logged out successfully"}

Utilizador Atual

Método: GET | Endpoint: /api/me | Autenticação: Requerida

Resposta 200: devolve id, name, email, role do utilizador autenticado.

6.4 Recursos

MÉTODO	ENDPOINT	PERFIL	DESCRIÇÃO
GET	/api/resources	Admin e Colaborador	Lista recursos ativos
GET	/api/resources/all	Admin	Lista todos os recursos (ativos e inativos)
GET	/api/resources/{id}	Admin e Colaborador	Obtém um recurso por ID
POST	/api/resources	Admin	Cria um novo recurso
PUT	/api/resources/{id}	Admin	Edita um recurso ou altera is_active
DELETE	/api/resources/{id}	Admin	Elimina um recurso

Campos do body (POST/PUT): name (string, obrigatório), type (Enum: space/equipment/vehicle/other, obrigatório), description (string, opcional), is_active (Boolean, apenas PUT).

6.5 Requisições

MÉTODO	ENDPOINT	PERFIL	DESCRIÇÃO
GET	/api/requests	Admin e Colaborador	Admin vê todas; Colaborador vê as suas
POST	/api/requests	Colaborador	Cria uma nova requisição
GET	/api/requests/{id}	Admin e Colaborador	Obtém uma requisição por ID
PATCH	/api/requests/{id}/cancel	Admin e Colaborador	Cancela uma requisição

Campos do body (POST): resource_id (integer, obrigatório), start_date (datetime, obrigatório, não pode ser no passado), end_date (datetime, obrigatório, após start_date), observations (string, opcional).

6.6 Aprovação

MÉTODO	ENDPOINT	PERFIL	DESCRIÇÃO
PATCH	/api/requests/{id}/approve	Admin	Aprova uma requisição pendente
PATCH	/api/requests/{id}/reject	Admin	Rejeita uma requisição pendente

PATCH	/api/requests/{id}/complete	Admin	Conclui uma requisição aprovada
GET	/api/dashboard	Admin	Devolve estatísticas globais do sistema
GET	/api/requests?status=pending	Requerida	Filtra requisições por estado
GET	/api/resources/all?type=space&is_active=true	Admin	Filtra recursos por tipo e estado

Campos do body (approve/reject): admin_notes (string, opcional para aprovação, obrigatório para rejeição).

7. Segurança

A segurança do sistema foi implementada em múltiplas camadas, garantindo que nenhuma operação crítica depende exclusivamente da interface do utilizador para ser protegida. Todas as regras de acesso e validação são aplicadas no backend, independentemente do que o frontend apresenta ou omite.

7.1 Autenticação e Autorização

O sistema utiliza Laravel Sanctum para autenticação baseada em tokens. Após login bem-sucedido com email e password, o servidor gera um token único associado ao utilizador. Este token deve ser enviado em todos os pedidos subsequentes no header `Authorization: Bearer <token>`. O token é invalidado no momento do logout, garantindo que sessões terminadas não podem ser reutilizadas.

O sistema implementa controlo de acesso baseado em perfis (RBAC - Role-Based Access Control). Existem dois perfis: Administrador e Colaborador. A verificação de perfil é feita exclusivamente no backend através de verificações nos controllers e Form Requests. O frontend oculta elementos de interface conforme o perfil, mas esta é apenas uma medida de usabilidade, não de segurança.

<i>Operação</i>	<i>Perfil permitido</i>
<i>Login / Logout</i>	Administrador e Colaborador
<i>Consultar recursos ativos</i>	Administrador e Colaborador
<i>Criar / Editar / Eliminar recursos</i>	Apenas Administrador
<i>Criar requisição</i>	Apenas Colaborador
<i>Consultar requisições</i>	Colaborador (as suas) / Administrador (todas)
<i>Cancelar requisição</i>	Autor da requisição ou Administrador
<i>Aprovar / Rejeitar / Concluir</i>	Apenas Administrador

7.2 Proteção de Dados

7.2.1 Passwords

Todas as passwords são armazenadas com hash bcrypt, nunca em texto simples. O Laravel aplica este hash automaticamente através do cast hashed definido no modelo User, garantindo que mesmo em caso de acesso não autorizado à base de dados, as passwords não são legíveis.

7.2.2 Variáveis de ambiente

Todas as configurações sensíveis, credenciais da base de dados, chave da aplicação e configuração de CORS, são definidas em ficheiro .env, que nunca é versionado no repositório Git. O repositório inclui um ficheiro .env.example com a estrutura necessária mas sem valores reais, para facilitar a instalação em novos ambientes.

7.2.3 Exposição de dados

O campo password está definido como hidden no modelo User e nunca é incluído nas respostas da API, mesmo que o campo exista na base de dados.

7.3 Prevenção de Vulnerabilidades

<i>Vulnerabilidade</i>	<i>Medida implementada</i>
<i>SQL Injection</i>	Mitigado pelo uso do ORM Eloquent, nunca são construídas queries por concatenação de strings
<i>XSS (Cross-Site Scripting)</i>	O Angular faz escape automático de todos os valores interpolados nos templates
<i>CSRF</i>	Não aplicável em APIs stateless com autenticação por token, não existe sessão baseada em cookies
<i>Acesso não autorizado a recursos de terceiros</i>	Verificação de ownership em todas as operações sensíveis, o backend compara o user_id da requisição com o utilizador autenticado
<i>Dados inválidos</i>	Validação centralizada em Form Requests dedicados para cada operação, o controller só executa se os dados já estiverem validados

8. Docker

O projeto utiliza Docker e Docker Compose para containerização e orquestração de todos os serviços que compõem o sistema. Esta abordagem garante que o ambiente de desenvolvimento é totalmente reproduzível em qualquer máquina, eliminando problemas de compatibilidade de versões ou configurações locais.

8.1 Arquitetura de Containers

O sistema é composto por três containers que comunicam entre si através de uma rede interna Docker:

<i>Container</i>	<i>Tecnologia</i>	<i>Porto</i>	<i>Função</i>
<i>requisicoes_backend</i>	Laravel	8000	API REST - backend da aplicação
<i>requisicoes_frontend</i>	Angular	4200	SPA - interface do utilizador
<i>requisicoes_mysql</i>	MySQL 8	3306	Base de dados relacional

Apenas os portos 4200 e 8000 estão expostos ao host. O MySQL é acessível apenas internamente entre containers, nunca diretamente do exterior.

9. Guia de Instalação

9.1 Pré-requisitos

Antes de instalar o sistema, garantir que as seguintes ferramentas estão instaladas na máquina:

FERRAMENTA	VERSÃO MÍNIMA	COMO VERIFICAR
DOCKER	24.0	<code>docker --version</code>
DOCKER COMPOSE	2.0	<code>docker compose version</code>
GIT	Qualquer	<code>git --version</code>

9.2 Passos de Instalação

1. Clonar o repositório

- `git clone https://github.com/bpedro08/gestao-requisicoes.git`
- `cd gestao-requisicoes`

2. Configurar as variáveis de ambiente

- `cp backend/.env.example backend/.env`
- Abrir o ficheiro `backend/.env` e confirmar os seguintes valores:
- `APP_KEY=` ← gerado no passo 5
- `DB_HOST=mysql`
- `DB_DATABASE=requisicoes`
- `DB_USERNAME=root`

- DB_PASSWORD=secret

3. Iniciar os containers

```
docker compose up -d --build
```

Aguardar que todos os containers estejam em execução. Na primeira vez este processo pode demorar alguns minutos.

4. Executar as migrations e o seeder

```
docker exec -it requisicoes_backend php artisan migrate:fresh --seed
```

Este comando cria todas as tabelas e insere os dados de teste iniciais.

5. Gerar a chave da aplicação

```
docker exec -it requisicoes_backend php artisan key:generate
```

9.3 Verificação da Instalação

Após os passos acima, verificar que os seguintes endereços estão acessíveis no browser:

<i>Serviço</i>	<i>URL</i>	<i>Resultado esperado</i>
<i>Frontend</i>	http://localhost:4200	Página de login do sistema
<i>API Backend</i>	http://localhost:8000/api	Resposta JSON
<i>Health check</i>	http://localhost:8000/up	OK

9.4 Credenciais de Teste

O seeder cria automaticamente as seguintes contas:

<i>Perfil</i>	<i>Email</i>	<i>Password</i>
<i>Administrador</i>	admin@company.com	password
<i>Colaborador</i>	jon@winterfell.com	thewall
<i>Colaborador</i>	walt@bluecrystal.com	heisenberg

Nota: estas credenciais são exclusivamente para ambiente de desenvolvimento e demonstração.

9.5 Paragem do Sistema

docker compose down ← para os containers, preserva os dados

docker compose down -v ← para os containers e apaga todos os dados

10. Conclusão

Síntese do Trabalho Desenvolvido

O presente projeto consistiu no desenvolvimento de um sistema de Gestão de Requisições Internas, denominado ReqManager, que permite a colaboradores de uma organização solicitar a utilização de recursos compartilhados e a administradores gerir esses pedidos de forma centralizada e controlada.

O sistema foi desenvolvido de raiz, cobrindo todas as fases do ciclo de desenvolvimento de software, desde a análise e especificação de requisitos, passando pela modelação e arquitetura, até à implementação, testes e documentação, simulando um contexto real de desenvolvimento em ambiente empresarial.